

4.3.1 Usage patterns for Option

Partial functions abound in programming, and `Option` (and the `Either` data type that we'll discuss shortly) is typically how this partiality is dealt with in FP. You'll see `Option` used throughout the Scala standard library, for instance:

- Map lookup for a given key (<http://mng.bz/ha64>) returns `Option`.
- `headOption` and `lastOption` defined for lists and other iterables (<http://mng.bz/Pz86>) return an `Option` containing the first or last elements of a sequence if it's nonempty.

These aren't the only examples—we'll see `Option` come up in many different situations. What makes `Option` convenient is that we can factor out common patterns of error handling via higher-order functions, freeing us from writing the usual boilerplate that comes with exception-handling code. In this section, we'll cover some of the basic functions for working with `Option`. Our goal is not for you to attain fluency with all these functions, but just to get you familiar enough that you can revisit this chapter and make progress on your own when you have to write some functional code to deal with errors.

BASIC FUNCTIONS ON OPTION

`Option` can be thought of like a `List` that can contain at most one element, and many of the `List` functions we saw earlier have analogous functions on `Option`. Let's look at some of these functions.

We'll do something slightly different than in chapter 3 where we put all our `List` functions in the `List` companion object. Here we'll place our functions, when possible, inside the body of the `Option` trait, so they can be called with the syntax `obj.fn(arg1)` or `obj fn arg1` instead of `fn(obj, arg1)`. This is a stylistic choice with no real significance, and we'll use both styles throughout this book.²

This choice raises one additional complication with regard to variance that we'll discuss in a moment. Let's take a look.

Listing 4.2 The Option data type

```

Don't
evaluate
ob unless
needed.
trait Option[+A] {
  def map[B](f: A => B): Option[B]
  def flatMap[B](f: A => Option[B]): Option[B]
  def getOrElse[B >: A](default: => B): B
  def orElse[B >: A](ob: => Option[B]): Option[B]
  def filter(f: A => Boolean): Option[A]
}

```

Apply f if the `Option` is not `None`.

Apply f , which may fail, to the `Option` if not `None`.

The `B >: A` says that the `B` type parameter must be a supertype of `A`.

Convert `Some` to `None` if the value doesn't satisfy f .

² In general, we'll use this object-oriented style of syntax where possible for functions that have a single, clear operand (like `List.map`), and the standalone function style otherwise.

There is some new syntax here. The `default: => B` type annotation in `getOrElse` (and the similar annotation in `orElse`) indicates that the argument is of type `B`, but won't be evaluated until it's needed by the function. Don't worry about this for now—we'll talk much more about this concept of *non-strictness* in the next chapter. Also, the `B >: A` type parameter on the `getOrElse` and `orElse` functions indicates that `B` must be equal to or a *supertype* of `A`. It's needed to convince Scala that it's still safe to declare `Option[+A]` as covariant in `A`. See the chapter notes for more detail—it's unfortunately somewhat complicated, but a necessary complication in Scala. Fortunately, fully understanding subtyping and variance isn't essential for our purposes here.



EXERCISE 4.1

Implement all of the preceding functions on `Option`. As you implement each function, try to think about what it means and in what situations you'd use it. We'll explore when to use each of these functions next. Here are a few hints for solving this exercise:

- It's fine to use pattern matching, though you should be able to implement all the functions besides `map` and `getOrElse` without resorting to pattern matching.
- For `map` and `flatMap`, the type signature should be enough to determine the implementation.
- `getOrElse` returns the result inside the `Some` case of the `Option`, or if the `Option` is `None`, returns the given default value.
- `orElse` returns the first `Option` if it's defined; otherwise, it returns the second `Option`.

USAGE SCENARIOS FOR THE BASIC OPTION FUNCTIONS

Although we can explicitly pattern match on an `Option`, we'll almost always use the above higher-order functions. Here, we'll try to give some guidance for when to use each one. Fluency with these functions will come with practice, but the objective here is to get some basic familiarity. Next time you try writing some functional code that uses `Option`, see if you can recognize the patterns these functions encapsulate before you resort to pattern matching.

Let's start with `map`. The `map` function can be used to transform the result inside an `Option`, if it exists. We can think of it as proceeding with a computation on the assumption that an error hasn't occurred; it's also a way of deferring the error handling to later code:

```
case class Employee(name: String, department: String)

def lookupByName(name: String): Option[Employee] = ...

val joeDepartment: Option[String] =
  lookupByName("Joe").map(_.department)
```

Here, `lookupByName("Joe")` returns an `Option[Employee]`, which we transform using `map` to pull out the `Option[String]` representing the department. Note that we don't need to explicitly check the result of `lookupByName("Joe")`; we simply continue the computation as if no error occurred, inside the argument to `map`. If `employeesByName.get("Joe")` returns `None`, this will abort the rest of the computation and `map` will not call the `_.department` function at all.



`flatMap` is similar, except that the function we provide to transform the result can itself fail.



EXERCISE 4.2

Implement the `variance` function in terms of `flatMap`. If the mean of a sequence is `m`, the variance is the mean of `math.pow(x - m, 2)` for each element `x` in the sequence. See the definition of variance on Wikipedia (<http://mng.bz/0Qsr>).

```
def variance(xs: Seq[Double]): Option[Double]
```

As the implementation of `variance` demonstrates, with `flatMap` we can construct a computation with multiple stages, any of which may fail, and the computation will abort as soon as the first failure is encountered, since `None.flatMap(f)` will immediately return `None`, without running `f`.